

Automatic planning engine ("auto pilot"): what has been tried, why it loops, and how to finish it

Main repository: [himanshujena64-dot/segment-wise-enquiry](#), plus five earlier repositories of the same app | App: SAP MRP Engine (Streamlit, single file *mrp_req_version (1).py*) | Period reviewed: 22 Mar 2026 to 30 Sep 2026 (6 repositories) | Prepared 1 Oct 2026

Scope note on the term "auto pilot"

The phrase "auto pilot" does not appear in any commit, pull request, issue or source line of this repository, nor in any of the five earlier repositories that hold previous versions of the same app (listed in section 2). This report therefore treats "auto pilot" as the **automatic planning engine** of the app: the part that, without manual steps, explodes BOMs, allocates import-part stock to FG models with a linear programme (LP), rolls sets month by month against the Requirement sheet, and adds PO arrivals week by week. That is the only piece of this project that has been under continuous correction for the last month.

One more repository in your account, **Zyugantax/Newtrail** (private, pushed today), could not be opened from this session: the request to attach it was denied by the session's permission policy. If "auto pilot" lives there, this report does not cover it and that repository needs to be attached to a session first.

1. Executive summary

- **Volume:** 241 commits across six repositories between 22 Mar and 30 Sep 2026, all on the same "SAP MRP Engine" Streamlit app. Four bursts: 27 to 28 Mar (about 80 whole-file deletes and re-uploads in one repo), 17 to 28 Apr (two new repos, LP allocation first appears), 8 to 22 May (40 edits here, file bouncing between 833 and 2,186 lines on a single day), 26 to 30 Jun (two more repos, a 2,520-line fork), then 28 to 30 Sep (15 Claude pull requests here). The Sep 28 restart was built on the **May 22** version, so whatever was improved in the June fork was left behind.
- **Pattern:** every pull request fixed what was visible on screen after the previous one. At least six of the fifteen September changes reverse or rework an earlier change made less than 24 hours before, and the same pattern (replace the whole file, see what breaks, replace again) is visible in March, April, May and June (details in section 5). The planning rule for Alt BOMs was changed three times in two days.
- **Root cause of the loop:** there has never been (a) a written definition of the correct answer, (b) a reference dataset with hand-checked expected results saved in the repository, or (c) a single automated test committed. Each pull request description says tests were run, but those scripts lived only inside the session and were discarded. So every change was judged by eye on a screenshot, and every change to the engine silently moved the numbers, which triggered the next request.
- **Have you been misguided?** Partly. Each individual change was reasonable and did what was asked. The misguidance is structural: the work was driven by screen appearance (columns, tabs, pinned headers, Excel sheets, render method) while the engine logic underneath kept being changed as a side effect, with nothing pinning down the expected numbers. That is exactly the "problem after problem" loop you describe.
- **What to do:** stop feature and layout requests for one iteration. Freeze the planning rules on one page, build one small golden dataset with expected output computed by hand, put the engine in its own module with a test that reproduces that output, and only then resume changes, one per pull request, each gated by that test. Section 7 gives the concrete steps and an order of work.

2. The six-month lineage: one app, six repositories

Every repository below contains the same Streamlit app (page title "SAP MRP Engine"). Each new repository was started by uploading a copy of the file from the previous one. Similarity was measured line by line between the main files.

Repository	Active	Commits	What it holds and what happened
Material-Requirement-Cal.	22 Mar to 18 Apr	124	First version: L1-L4 BOM explosion, phantom handling, Alt-aware joins. 81 of the commits are "Delete App.py" / "Add files via upload" pairs, most of them on 28 Mar alone. Has the only README. app (1).py 976 lines; app-withPW.py adds a password.
Material-Requirement-Cal-with-Display-Option	17 Apr to 28 Apr (+ devcontainer 25 Sep)	19	Copy of the above (76 percent of functions shared) plus the first LP allocation (scipy linprog over segments). app (6).py 1,091 lines. Revisited on 25 Sep, three days before the September push here.
Modified-file	20 Apr to 25 Apr	8	Another copy, app (8).py 918 lines, different direction (only 20 percent of functions shared). Abandoned after five days.
segment-wise-enquiry (this repo)	8 May to 22 May, then 28 to 30 Sep	78	Starts at 1,310 lines on 8 May. Aging Projection added 11 May; Segment Wise page 13 to 14 May; month-wise views 18 May; Production Plan 20 May. 2,259 lines on 22 May. Silent for four months, then the September burst.
LPlanning	26 May to 30 Jun	7	Only a 195-line paste-in block (Excel template generator) with the instruction "paste this whole block into your app". Not a runnable app.
LPPLANINGHJENA	30 Jun	5	A 2,520-line fork of the May 22 version (81 percent similar) with further Segment Wise work, uploaded in one morning. Never carried forward: the 28 Sep file here is 92 percent similar to May 22 and only 76 percent to this June version.
Zyugantax/Newtrail (private)	pushed 1 Oct	?	Not accessible from this session; see scope note.

What this lineage shows

- **The app was restarted from a copy at least five times.** Every restart lost the fixes made in the branch it did not come from. The June work is the clearest example: a month of effort that the September restart never saw.
- **The LP "auto" allocation has existed since April** and has been re-implemented in at least three files (app (6).py, the May version here, and the June fork) with no shared test or definition. Each re-implementation re-decided the same questions.
- **Whole-file replacement is the editing method** throughout: on 28 Mar about 80 delete-and-upload commits; on 13 May the file went 1,942 to 833 to 1,739 to 1,718 to 1,942 to 2,186 to 1,793 lines in one afternoon; on 28 Sep it went 2,259 to 729 to 2,430 to 2,647 in one morning. Those swings mean several copies were alive at once and were being pasted over each other.
- **Commit messages carry no information** ("Update ...", "Add files via upload"), so even you cannot reconstruct today what any given version fixed or broke. The September pull requests are the first time the history explains itself.

3. Timeline of what was tried (May and September, this repository)

All dates 2026. PR = pull request on GitHub, all opened from branch *claude/zealous-fermi-umcesf* and merged by you within minutes of opening.

When	Who	What was tried	Nature
27 Mar to 28 Apr	You	Earlier repositories (see section 2): first MRP explosion app, then first LP segment allocation in app (6).py. Three repos created and left within six weeks.	Copies and re-uploads
8 to 22 May	You	40 direct edits to the app file ("Update mrp_req_version (1).py"). Pages at that point: Home, Upload, Run MRP, Segment Wise Available, Aging Projection, Production Plan, Settings. 2,259 lines.	Manual, in GitHub web editor
13 May	You	Seven uploads in one afternoon; the file swung between 833 and 2,186 lines. The Segment Wise page first appears in the 16:51 upload and is gone again in the 17:13 one, then returns on 14 May.	Version swapping
26 to 30 Jun	You	Work continues in LPlanning (template paste-in) and LPPLANINGHJENA (2,520-line fork). Not merged back here.	Parallel fork
25 Sep	You	Devcontainer added to the April repo Material-Requirement-Cal-with-Display-Option: the older LP version was reopened three days before the September burst here.	Revisit of old copy
28 Sep (a.m.)	You	Four direct edits. The first replaced the file with a 729-line version (Upload, Production Plan, Settings only), the second brought a 2,430-line version back, then +217 lines (flexible column-name detection, sheet-name resolver, template info box), then +8. Final: 2,647 lines.	Manual version swapping
28 Sep	PR #1	New Import Shortage & Coverage page: PO file upload, month-of-ETA placement, per-part status (new PO / expedite / covered), Excel export.	+319 lines
28 Sep	PR #2	Segment page: month-wise sets vs requirement. Defined Available, Req, WK01-WK04 arrivals, Balance, Shortfall, Remarks. Alt BOM = alts with demand in Requirement sheet. Added "Max Sets" vs "Aligned to requirement" basis.	+272 lines
29 Sep	PR #3	Large rework: Segment-wise tab, Model-wise (sets + components) tab replacing the plain Model-wise table, Component shortage tab, interchangeable PCB groups pooling stock, week-wise PO template, leading-zero code matching.	+415 / -59 lines
29 Sep	PR #4	Save uploads to disk so restarts and redeploys on the hosted trial do not lose files.	+74 lines
29 Sep	PR #5	Second Excel download button placed above the tabs (5 sheets).	+12 lines
29 Sep	PR #6	Restored the FG-rows-only Model-wise table that PR #3 had removed, as its own tab.	Reversal of #3
29 Sep	PR #7	Model + components: new column set, removed the complete-set rows, filter default set to "Short only".	Rework of #3
29 Sep	PR #8	One-line fix: PR #7's default hid every row when nothing was short, so the tab was empty. Default changed to "All".	Fix of #7
29 Sep	PR #9	Fixed column widths and pinned columns so months stay visible on narrow screens.	Layout
29 Sep	PR #10	Hosted Streamlit ignored the HTML styling of PR #6 to #9; tables re-rendered through components.html.	Fix of #9 on host
30 Sep	PR #11	Excel cut from 5 to 3 sheets; set-level views now stock only (PO arrivals removed from sets, kept only on components); Model column was always blank and fixed; render switched again to st.iframe.	Reversal of #2/#3 arrivals; rework of #10
30 Sep	PR #12	Description and Supplier columns, optional Supplier Master upload, two shortfall columns (before / after arrival).	+94 / -32 lines
30 Sep	PR #13	RM Group column; Excel header row made filterable (merged headers had blocked AutoFilter).	Rework of #5/#11 Excel
30 Sep	PR #14	Engine change: one variant per FG x IDU Alt from the BOM, ODU alt matched by number, requirement split per alt.	Alt rule v2
30 Sep	PR #15	Engine change 1.5 hours later: Requirement sheet becomes the master for alts; BOM-only alts ignored.	Alt rule v3

4. How the engine works today (as merged on 30 Sep)

- **Inputs:** BOM (multi-level, Alt column), Requirement & Stock workbook (month columns), Segment & Import Part file (FG, Segment, IDU, Compatible ODU, Category / RM Group, Interchange Group), optional Import PO file (ETA / ETD, qty), optional Supplier Master.
- **run_segment:** explodes IDU and ODU BOMs down to the import parts, builds one variant per FG x Alt (alts taken from the Requirement sheet since PR #15), pools interchangeable parts, then solves one LP: maximise total sets across all variants subject to stock. The LP result is floored to integers.
- **segment_monthwise:** opening sets of month 1 = LP result from stock; each later month opens with the previous balance. For every week with PO arrivals it adds the arrivals to supply and re-solves the LP, never taking back sets already counted. Balance = $\max(0, \text{opening} + \text{arrivals} - \text{req})$; Shortfall = $\max(0, \text{req} - \text{opening} - \text{arrivals})$. "Aligned to requirement" uses a second LP with one variable per FG and month, weighted so earlier months are served first.
- **component_monthwise:** a pieces-level ledger per FG x BOM header x component, with before-arrival and after-arrival shortfall.
- **Display:** five tabs on the Segment page, HTML tables rendered inside an iframe, two Excel downloads with merged month headers.

Technical observations that matter for correctness

- **Inconsistent basis between tabs (from PR #11).** Segment-wise and Model-wise (sets) exclude PO arrivals; Model + components includes them. The same FG can show "Short" in one tab and "Covered by arrival" in another. This is a design decision made in a hurry, not a bug, but it will be read as a bug by anyone using the sheets.
- **The LP objective is "most sets in total", not a business priority.** With shared parts the solver gives stock to whichever models consume least per set, and when several allocations tie the choice is arbitrary. Two runs with slightly different data can shift sets between models for no planning reason. Nobody has decided what the correct priority is (by segment, by requirement month, by margin, pro rata).
- **Stock is treated as fully available in month 1.** Month 2 only ever receives the carry-forward balance plus arrivals. If month 1 requirement is small, month 1 "Available" looks enormous and the shortfall only appears later. That matches the planning-sheet convention you asked for in PR #2, but it must be stated and agreed, because it drives every number.
- **Flooring after a continuous LP loses sets** (several models each lose a fractional set that together would have made whole sets).
- **Alt BOM rule changed three times** (PR #2, #14, #15) and each change changes which rows exist and how requirement is split. There is no written rule to test against.
- **Code shape:** one 3,825-line file with 87 functions, UI and engine mixed, nine silent *except: pass* blocks that hide data problems, no tests folder, no sample data, no README, and Streamlit unpinned ($\geq 1.35.0$) which is why the hosted app rendered differently from the sessions in PR #10 and #11.

5. Are we in a loop, and why

Yes, and it predates September. The September pull requests only make the loop visible because they are the first changes with descriptions:

Change	Undone or reworked by
PR #3 removed the plain Model-wise table	PR #6 restored it the same morning
PR #7 set the component filter default to "Short only"	PR #8, 11 minutes later, changed it back because the tab was empty
PR #6 to #9 styled tables with st.markdown HTML	PR #10 re-rendered with components.html because the hosted app ignored the styling; PR #11 switched again to st.iframe
PR #2 and #3 put weekly PO arrivals into the set tables	PR #11 removed arrivals from set tables (stock only), keeping them on components
PR #5 added a 5-sheet Excel	PR #11 cut it to 3 sheets; PR #13 rebuilt the headers because merged cells blocked filtering
PR #14 derived Alt variants from the BOM	PR #15 overrode it 90 minutes later: Requirement sheet is master

Why it keeps happening

- **No definition of done.** Requests were phrased as screen outcomes ("add column", "pin header", "3 sheets"). The engine's expected numbers were never written down, so there was no way to notice when a layout change also changed a result, or when an engine change broke an earlier agreed behaviour.
- **Tests were never kept.** Every PR body describes AppTest or headless-browser checks. None of that is in the repository. Each new session started from zero and could only re-verify what it was told about, so regressions in untouched behaviour were invisible.
- **Decisions were taken inside implementation.** The allocation priority, the arrivals basis, the Alt rule and the month-1 convention are planning policy. They were each decided ad hoc inside a code change and then re-decided when the output looked wrong.
- **Environment drift.** The hosted app and the sessions ran different Streamlit versions. Two PRs (#10, #11) exist only because of that.
- **Parallel forks never merged.** March, April, May and June each produced a separate repository. Fixes made in one do not exist in the others, so the same defect is met again in the next copy and feels like a problem that "keeps coming back".
- **Version juggling by hand.** On 28 Sep the file was swapped from 2,259 to 729 to 2,430 lines within hours, which means several local copies of the app existed. Any fix applied to the wrong copy is lost.
- **Large, mixed pull requests.** PR #3 changed 415 lines across engine, template, matching and UI at once. When something looked wrong afterwards, there was no way to tell which part caused it.

Verdict: the individual fixes were not wrong. The process was. Continuing to request the next visible fix will produce PR #16, #17 and so on indefinitely, because the thing being corrected (the numbers) is not what is being checked (the screen).

6. What the goal actually is

From the PR descriptions and the May code, the goal is a planner that, from BOM, Requirement, Stock and PO files, tells you per segment and per FG model, month by month, how many sets can be built, which import component limits it, whether an incoming PO closes the gap and in which week, and lets you download that as a filterable Excel. The app already contains every one of those features. What is missing is confidence that the numbers are right and stay right. The goal for the next phase is therefore **trust**, not features.

7. What exactly to do now

Do these in order. Steps 0 to 4 need no new features and should take one or two sessions. Do not request any layout or column change until step 4 is green.

#	Step	Detail
0	Pick one repository and archive the rest.	This repository has the most complete engine (September merges). Archive Material-Requirement-Cal., Material-Requirement-Cal-with-Display-Option, Modified-file, LPlanning and LPPLANNINGHJENA on GitHub (Settings, Archive) so they cannot be edited by mistake. Before archiving LPPLANNINGHJENA, diff its Segment Wise code against this repo and list anything useful it had that was lost; bring that in as one pull request.
1	Write the planning rules on one page (docs/PLANNING_RULE S.md).	Decide and write down: (a) Alt rule: which alts form a row and where requirement comes from (keep PR #15 unless you disagree); (b) allocation priority when parts are shared: pro rata, by segment priority, by earliest requirement month, or "max sets" with a stated tie-break; (c) whether set tables include PO arrivals (make all tabs use the same basis); (d) month-1 convention: all stock available in month 1, carry-forward thereafter; (e) week bucketing of ETA (1-7, 8-14, 15-21, 22+); (f) what a blank ETA means. This page is the contract every future change is checked against.
2	Create one golden dataset and its expected answer by hand.	A small but real-shaped set: 2 segments, 3 FGs, one IDU with 2 alts, one shared ODU, 6 to 8 import parts, 3 months of requirement, 3 or 4 PO lines across different weeks, one interchange group. Compute the expected Segment-wise, Model-wise and component ledgers in Excel by hand using the rules from step 1. Commit both under tests/golden/. Several PR bodies describe exactly such datasets; they just were never saved.
3	Split the engine from the screen.	Move run_mrp_engine, run_segment, segment_monthwise, component_monthwise, component_shortage, import_arrivals_by_week, run_import_coverage and the loaders into mrp_engine.py with no Streamlit calls (return warnings as lists instead of st.warning). Keep app.py as the Streamlit shell. Rename away from "mrp_req_version (1).py". This is a move, not a rewrite; behaviour must not change.
4	Add tests that run the engine on the golden data and compare to the expected answer.	pytest, run with one command, committed to the repo and wired to GitHub Actions so every pull request shows green or red. Also add a devcontainer/postCreate step so sessions can run it. From this point a pull request that changes any number must either fail the test (and be fixed) or come with a deliberate update to the expected answer and an edit to the rules page.
5	Fix the known inconsistencies against the rules page.	Unify the arrivals basis across tabs (from step 1c). Replace the nine silent except: pass with warnings surfaced on the page. Decide integer handling (floor per model vs. integer LP). Pin streamlit== in requirements.txt so hosted and local look identical.
6	Validate once with real data on the hosted app, then sign off.	Upload your actual BOM, Requirement and PO files, pick 3 FGs and check them by hand against the Model + components tab. Record the result in the rules page. That is the definition of "corrected".
7	Resume feature work under new rules.	One change per pull request. Every pull request must state: which rule it implements or changes, and that the golden test passes. Layout-only requests are fine, but they must not touch mrp_engine.py.

Decisions only you can make (needed for step 1)

- When two models compete for the same PCB, who gets it first? (This is the single most important unanswered question and the reason numbers keep "moving".)
- Should Segment-wise and Model-wise sets include PO arrivals, as they did before PR #11, or stay stock only?
- Is the Requirement sheet the master for Alt BOMs (PR #15), or should BOM-only alts also appear with zero requirement?
- Which Streamlit version runs on the hosted trial? (Needed to pin requirements.txt.)

What not to do

- Do not open another layout, column or Excel pull request before steps 1 to 4 are done. Each one restarts the loop.

- Do not start another repository or upload another copy of the file. One repository, one file name, branches and pull requests only.
- Do not accept "tests pass" in a pull request description unless the test file is part of the pull request.
- Do not change engine functions and UI in the same pull request.

Appendix: figures used in this report

Measure	Value
Repositories holding versions of this app	6 accessible (+1 private, not reviewed)
Commits across them, 22 Mar to 30 Sep 2026	241
Delete / re-upload commits in the March repo	81
Commits to main here, 28 to 30 Sep 2026	19 (15 PR merges, 4 direct edits)
Pull requests opened and merged	15, all self-merged within minutes, no review comments
GitHub issues	0
App file size, 8 May / 22 May / 30 Jun fork / 28 Sep / 30 Sep	1,310 / 2,259 / 2,520 / 2,647 / 3,825 lines
Sep 28 file similarity to May 22 / to June fork	92 percent / 76 percent
Net change since 22 May	+1,614 / -48 lines, one file
Top-level functions	87
Test files committed	0
Silent except: pass blocks	9
Changes reversed or reworked within 24 h	6 of 15
Alt BOM rule changes	3 (PR #2, #14, #15)